

FIT2004 - Algorithms and Data Structures

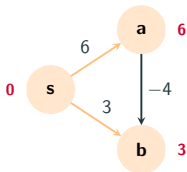
Seminar 7 - Dynamic Programming Graph Algorithms

Rafael Dowsley

Shortest path in graphs with negative weights

Shortest path in graphs with negative weights

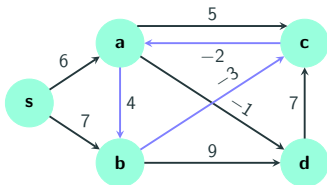
- What is the shortest distance from s to b in this graph?



- If Dijkstra's algorithm is executed on this graph, what will it output as being the shortest path from s to b ?
- Dijkstra's algorithm is not guaranteed to output the correct answer when there are negative weights.

Shortest path in graphs with negative weights

- What is the shortest distance from s to d in this graph?



- Not well-defined:**

- ▶ From s , it is possible to reach the negative cycle $a \rightarrow b \rightarrow c$, and from this cycle it is possible to reach d .
- ▶ Given any path P from s to d , it is possible to obtain an alternative path P' with smaller total weight than P : P' goes from s to the negative cycle, **include as many repetitions of the negative cycle as necessary**, and then reaches d from the negative cycle.

Bellman-Ford algorithm

- Bellman-Ford algorithm returns:
 - ▶ Shortest distances from the source s to all vertices in the graph if there are no negative cycles that are reachable from s .
 - ▶ An error if there is a negative cycle reachable from s (assuming that a final iteration is done to check this).
- Can be modified to return all valid shortest distances, and $-\infty$ for vertices which are affected by the negative cycle.

- If no negative cycles are reachable from node s , then for every node t that is reachable from s there is a shortest path from s to t that is simple (i.e., no nodes are repeated).
 - ▶ Cycles with positive weight cannot be part of a shortest path.
 - ▶ Given a shortest path that contains cycles of weight 0, the cycles can be removed to obtain an alternative shortest path that is simple.
- Note that any simple path has at most $|V| - 1$ edges.

Optimality of subpaths in shortest paths

- A fact from Week 5: If P is a shortest path $s \rightsquigarrow v$, and u is the last vertex on P before v , then the part of P from s to u (denoted Q) is also a shortest path.
- Assume that there is a path R from $s \rightsquigarrow u$ whose weight is smaller than that of Q .
- Let $T = R \cup (u, v)$.
- **Contradiction:** the weight of T is smaller than the weight of P .

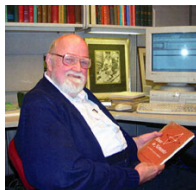


Bellman-Ford algorithm

One of the first applications of Dynamic Programming.



Richard Bellman



Lester Ford Jr.

Recursive formula for Dynamic Programming

- For a source node s , let $dist(i, v)$ denote the minimum weight of a $s \rightsquigarrow v$ path with at most i edges.
- Let P be an optimal path with at most i edges that achieves total weight $dist(i, v)$:
 - ▶ If P has at most $i - 1$ edges, then $dist(i, v) = dist(i - 1, v)$.
 - ▶ If P has exactly i edges and (u, v) is the last edge of P , then $dist(i, v) = dist(i - 1, u) + w(u, v)$, where $w(u, v)$ denotes the weight of edge (u, v) .
- Recursive formula for Dynamic Programming:

$$dist(i, v) = \min \left(dist(i - 1, v), \min_{u: (u, v) \in E} (dist(i - 1, u) + w(u, v)) \right)$$

Bellman-Ford original Dynamic Programming formulation

Bellman-Ford algorithm (original Dynamic Programming formulation)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V| - 1][1..|V|] = \infty$ 
3:    $dist[0][s] = 0$ 
4:   for  $i = 1$  to  $|V| - 1$  do
5:     for each vertex  $v$  in  $V$  do
6:        $dist[i][v] = \min (dist[i - 1][v], \min_{u:(u,v) \in E} (dist[i - 1][u] + w(u, v)))$ 
7:   return  $dist$            //  $dist[|V| - 1][1..|V|]$  contains the minimum distances
```

- What is the time complexity?

Bellman-Ford original Dynamic Programming formulation

Bellman-Ford algorithm (original Dynamic Programming formulation)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V| - 1][1..|V|] = \infty$ 
3:    $dist[0][s] = 0$ 
4:   for  $i = 1$  to  $|V| - 1$  do
5:     for each vertex  $v$  in  $V$  do
6:        $dist[i][v] = \min (dist[i - 1][v], \min_{u:(u,v) \in E} (dist[i - 1][u] + w(u, v)))$ 
7:   return  $dist$            //  $dist[|V| - 1][1..|V|]$  contains the minimum distances
```

- Time complexity: $\Theta(|V|^2 + |V| \cdot |E|)$. For connected graphs this simplifies to $\Theta(|V| \cdot |E|)$.
- Each iteration of the outer loop relaxes each edge once.

Bellman-Ford optimised version

- Often a more space-efficient version of Bellman-Ford algorithm is implemented.
- *i* is used just as a counter for the outer loop. In each of the $|V| - 1$ iterations of the outer loop, each edge $(u, v) \in E$ is relaxed once using the update rule:

$$dist(v) = \min(dist(v), (dist(u) + w(u, v)))$$

- In some cases, this version also provides a speed-up if an early stop condition is checked (but no improvement in the worst-case time complexity).

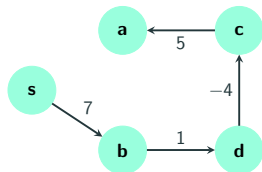
Bellman-Ford optimised version

Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do
6:     for each edge  $e = (u, v)$  in  $E$  do
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

- Time complexity: $\Theta(|V| \cdot |E|)$.
- Each iteration of the outer loop relaxes each edge once.

Bellman-Ford optimised version

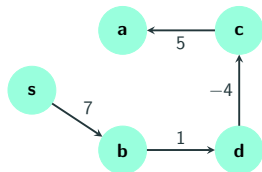


<i>i</i>	s	a	b	c	d
1	0	9	7	4	8
2	0	9	7	4	8
3	0	9	7	4	8
4	0	9	7	4	8

<i>i</i>	s	a	b	c	d
1	0	∞	7	∞	∞
2	0	∞	7	∞	8
3	0	∞	7	4	8
4	0	9	7	4	8

- If edges are relaxed in the order $s \rightarrow b$, $b \rightarrow d$, $d \rightarrow c$, $c \rightarrow a$, then we are done after 1 iteration.
- On the other hand, if edges are relaxed in the order $c \rightarrow a$, $d \rightarrow c$, $b \rightarrow d$, $s \rightarrow b$, then we need 4 iterations to get the right result.
- We will analyse the early stopping condition later on.

Bellman-Ford optimised version

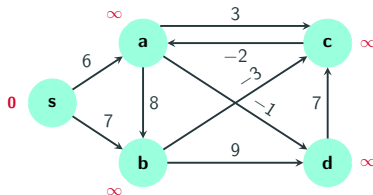


i	s	a	b	c	d
1	0	9	7	4	8
2	0	9	7	4	8
3	0	9	7	4	8
4	0	9	7	4	8

i	s	a	b	c	d
1	0	∞	7	∞	∞
2	0	∞	7	∞	8
3	0	∞	7	4	8
4	0	9	7	4	8

- For this space-efficient version of Bellman-Ford algorithm, there is a guarantee that after i iterations $dist[v]$ is no larger than the total weight of the shortest path $s \rightsquigarrow v$ that uses at most i edges.
- But there is no guarantee that these two values are equal after i iterations: depending on the order in which the edges are relaxed, the path $s \rightsquigarrow v$ that has weight $dist[v]$ could already contain more than i edges after the i -th iteration of the outer loop.

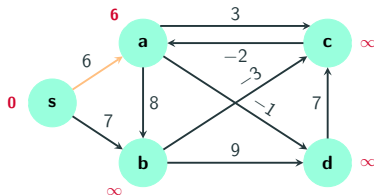
Bellman-Ford example



Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do
6:     for each edge  $e = (u, v)$  in  $E$  do
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Bellman-Ford example



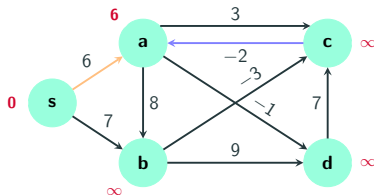
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $s \rightarrow a$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



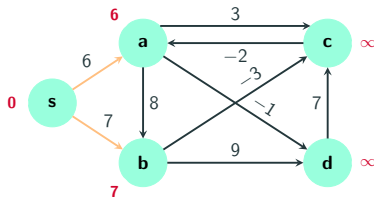
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $c \rightarrow a$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



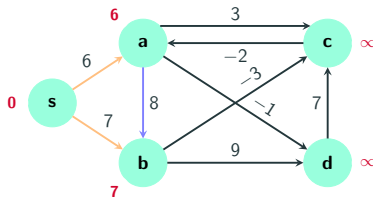
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $s \rightarrow b$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



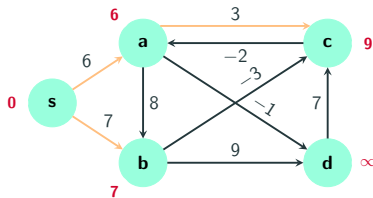
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $a \rightarrow b$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



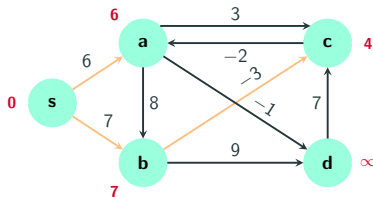
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $a \rightarrow c$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



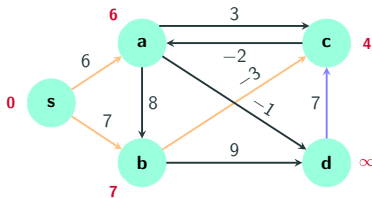
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $b \rightarrow c$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



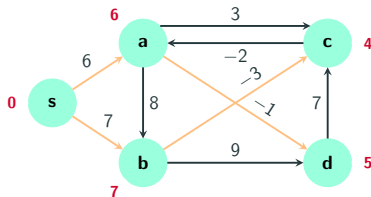
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $d \rightarrow c$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



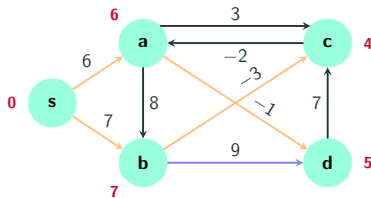
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $a \rightarrow d$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



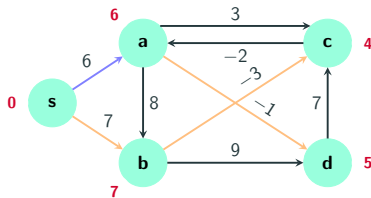
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 1$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $b \rightarrow d$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



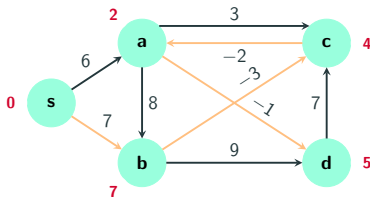
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $s \rightarrow a$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



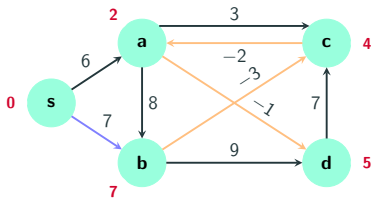
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $c \rightarrow a$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



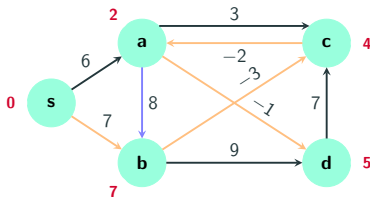
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $s \rightarrow b$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



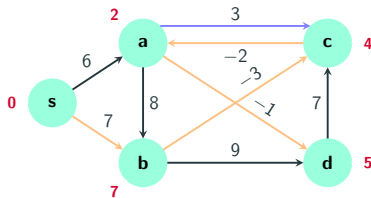
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $a \rightarrow b$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



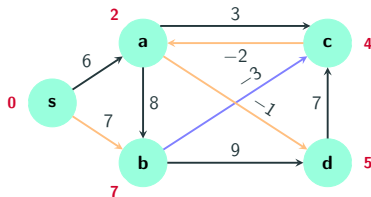
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $a \rightarrow c$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



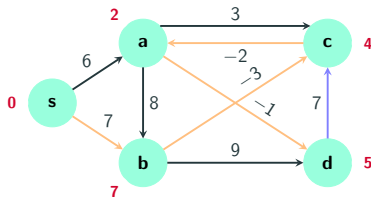
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $b \rightarrow c$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



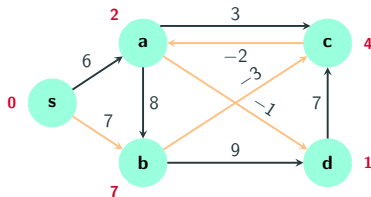
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $d \rightarrow c$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



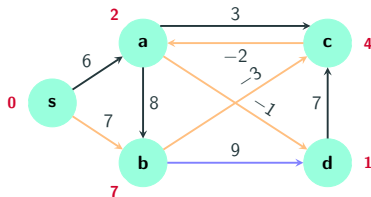
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $a \rightarrow d$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



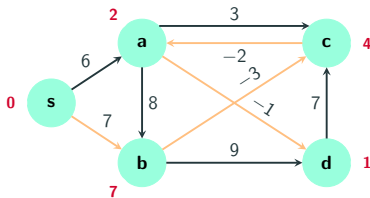
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E), s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do                                //  $i = 2$ 
6:     for each edge  $e = (u, v)$  in  $E$  do                    //  $b \rightarrow d$ 
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|], pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Bellman-Ford example



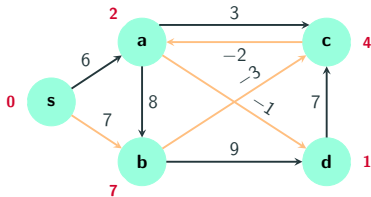
Bellman-Ford algorithm (optimised version)

```
1: function BELLMAN_FORD( $G = (V, E)$ ,  $s$ )
2:    $dist[1..|V|] = \infty$ 
3:    $pred[1..|V|] = \text{null}$ 
4:    $dist[s] = 0$ 
5:   for  $i = 1$  to  $|V| - 1$  do    // No updates while  $i = 3$ 
6:     for each edge  $e = (u, v)$  in  $E$  do
7:       if  $dist[u] + w(u, v) < dist[v]$  then
8:          $dist[v] = dist[u] + w(u, v)$ 
9:          $pred[v] = u$ 
10:  return  $dist[1..|V|]$ ,  $pred[1..|V|]$ 
```

Update order:

$s \rightarrow a$
 $c \rightarrow a$
 $s \rightarrow b$
 $a \rightarrow b$
 $a \rightarrow c$
 $b \rightarrow c$
 $d \rightarrow c$
 $a \rightarrow d$
 $b \rightarrow d$

Early stop condition



- **Early stop condition:** if nothing changes in one iteration of the outer loop, it is possible to stop the execution of the Bellman-Ford algorithm and output the current values.

Detecting reachable negative cycles

- **Idea: run one additional iteration of the outer loop.**
 - ▶ If this $|V|$ -th iteration reduces the distance estimate of a vertex v , **this means that there exists no simple shortest path $s \rightsquigarrow v$.**
 - ▶ There must be a negative cycle that is reachable from s and from which v is reachable.
- **How could we modify Bellman-Ford to determine which vertices have valid distances, and which are affected by negative cycles?**
 - ▶ Execute $|V|$ additional iterations of the outer loop, and for each node whose distance would be updated, just mark its distance as $-\infty$.

Detecting negative cycles

- Bellman-Ford algorithm only detects the existence of negative cycles if one such cycle is reachable from the source vertex.
- **How could we execute Bellman-Ford algorithm just once and determine if a given graph G has negative cycles?**
- Add one extra vertex to G and edges from it to every other vertex. Run Bellman-Ford algorithm with that new vertex as the source.

All-pairs shortest paths

All-pairs shortest paths

All-pairs shortest paths problem

Return shortest distances between all pairs of vertices in a graph.

For unweighted graphs:

- For each vertex v , call BFS with that node as the source.
- Time complexity: $\Theta(|V| \cdot (|V| + |E|)) = \Theta(|V|^2 + |V| \cdot |E|)$.
- For connected graphs this simplifies to $\Theta(|V| \cdot |E|)$.
- For dense graphs $|E| = \Theta(|V|^2)$, and so time complexity is $\Theta(|V|^3)$.

All-pairs shortest paths

All-pairs shortest paths problem

Return shortest distances between all pairs of vertices in a graph.

For weighted graphs with non-negative weights:

- For each vertex v , call Dijkstra's with that node as the source.
- Time complexity for connected graphs: $\Theta(|V| \cdot |E| \cdot \log |V|)$.
- For dense graphs the time complexity is $\Theta(|V|^3 \cdot \log |V|)$.

All-pairs shortest paths

All-pairs shortest paths problem

Return shortest distances between all pairs of vertices in a graph.

For weighted graphs allowing negative weights:

- For each vertex v , call Bellman-Ford with that node as the source.
- Time complexity: $\Theta(|V|^2 \cdot |E|)$.
- For dense graphs the time complexity is $\Theta(|V|^4)$.
- Can we do better?

Floyd-Warshall algorithm

Floyd-Warshall algorithm returns all-pairs shortest distances in $\Theta(|V|^3)$, even for graphs with negative weights.



Robert W. Floyd (Turing Award 1978)

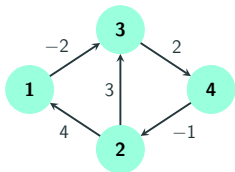


Stephen Warshall

Floyd-Warshall algorithm

- Algorithm based on Dynamic Programming.
- If the graph has a negative cycle, it can always be detected.
- This contrast with Bellman-Ford, which only detects negative cycles that are reachable from the source node.
- Main idea: for a graph without negative cycles, after the k -th iteration, $dist[u][v]$ contains the weight of the shortest path $u \rightsquigarrow v$ that only uses intermediate nodes from the set $\{1, \dots, k\}$.

Floyd-Warshall algorithm

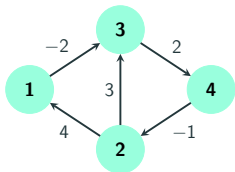


dist	1	2	3	4
1	∞	∞	∞	∞
2	∞	∞	∞	∞
3	∞	∞	∞	∞
4	∞	∞	∞	∞

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

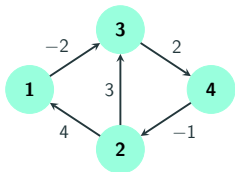


dist	1	2	3	4
1	0	∞	∞	∞
2	∞	0	∞	∞
3	∞	∞	0	∞
4	∞	∞	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

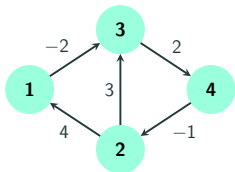


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

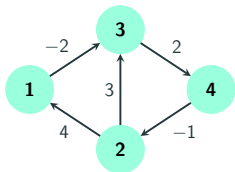


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                     //  $k = 1$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                   //  $u = 1$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                   //  $v = 1$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$            //  $0 = 0 + 0$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

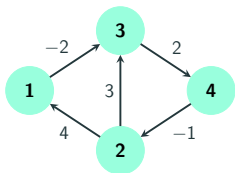


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                     //  $k = 1$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                               // No update while  $u = k = 1$ , check it!
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

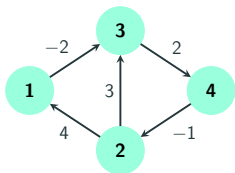


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                     //  $k = 1$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                   //  $u = 2$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                   //  $v = 1$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$            //  $4 = 4 + 0$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

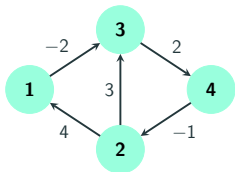


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                     //  $k = 1$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                   //  $u = 2$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                   //  $v = 2$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$            //  $0 < 4 + \infty$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

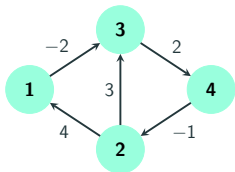


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                     //  $k = 1$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                   //  $u = 2$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                   //  $v = 3$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$            //  $3 > 4 - 2 = 2$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

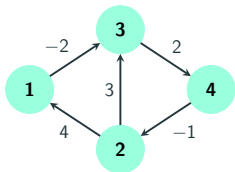


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                     //  $k = 1$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                   //  $u = 2$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                   //  $v = 3$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$  // Distance updated!
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

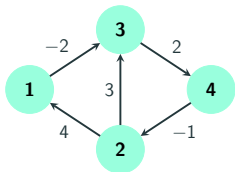


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                // Speeding things up:  $k = 2$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                //  $u = 4$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                //  $v = 1$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$  //  $\infty > -1 + 4 = 3$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

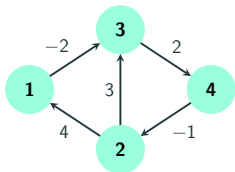


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	3	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                // Speeding things up:  $k = 2$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                //  $u = 4$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                //  $v = 1$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$  // Distance updated!
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

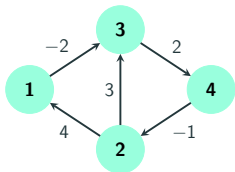


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	3	-1	∞	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                // Speeding things up:  $k = 2$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                //  $u = 4$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                //  $v = 3$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$  //  $\infty > -1 + 2 = 1$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

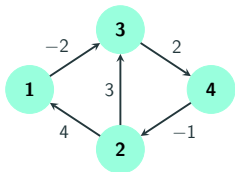


dist	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	3	-1	1	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                // Speeding things up:  $k = 2$ 
6:     for each vertex  $u = 1$  to  $|V|$  do                                //  $u = 4$ 
7:       for each vertex  $v = 1$  to  $|V|$  do                                //  $v = 3$ 
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$  // Distance updated!
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

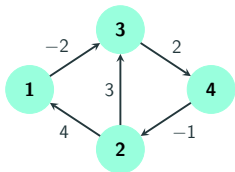


dist	1	2	3	4
1	0	∞	-2	0
2	4	0	2	4
3	∞	∞	0	2
4	3	-1	1	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                // In red the updates when  $k = 3$ 
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm



dist	1	2	3	4
1	0	-1	-2	0
2	4	0	2	4
3	5	1	0	2
4	3	-1	1	0

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do                                // In red the updates when  $k = 4$ 
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $dist[1..|V|][1..|V|] = \infty$ 
3:    $dist[v][v] = 0$  for all vertices  $v$ 
4:    $dist[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:          $dist[u][v] = \min(dist[u][v], dist[u][k] + dist[k][v])$ 
9:   return  $dist[1..|V|][1..|V|]$ 
```

- Time complexity: $\Theta(|V|^3)$
- Space complexity: $\Theta(|V|^2)$

Floyd-Warshall algorithm

```
1: function FLOYD_WARSHALL( $G = (V, E)$ )
2:    $\text{dist}[1..|V|][1..|V|] = \infty$ 
3:    $\text{dist}[v][v] = 0$  for all vertices  $v$ 
4:    $\text{dist}[u][v] = w(u, v)$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:         // Invariant: assuming that the shortest path  $u \rightsquigarrow v$  is well-defined,
            $\text{dist}[u][v]$  corresponds to the minimum distance from  $u$  to  $v$  considering only intermediate
           vertices 1 to  $k - 1$ .
9:          $\text{dist}[u][v] = \min(\text{dist}[u][v], \text{dist}[u][k] + \text{dist}[k][v])$ 
10:  return  $\text{dist}[1..|V|][1..|V|]$ 
```

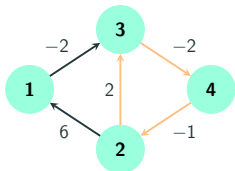
Correctness

- Invariant: assuming that the shortest path $u \rightsquigarrow v$ is well-defined, $dist[u][v]$ corresponds to the minimum distance from u to v considering only intermediate vertices 1 to $k - 1$.
- Base case $k = 1$ (i.e. no intermediate vertices allowed): true as $dist[1..|V|][1..|V|]$ is initialised based only on the adjacent edges.

- Invariant: assuming that the shortest path $u \rightsquigarrow v$ is well-defined, $\text{dist}[u][v]$ corresponds to the minimum distance from u to v considering only intermediate vertices 1 to $k - 1$.
- Inductive step:
 - ▶ Assume $\text{dist}[u][v]$ is the minimum distance of any path $u \rightsquigarrow v$ detouring through only vertices 1 to $k - 1$.
 - ▶ Adding the k -th vertex to the “detour pool” can only help if there is a shorter path detouring through k .
 - ▶ Otherwise, $\text{dist}[u][v]$ remains the minimum distance from u to v using intermediate vertices 1 to k .
 - ▶ If vertex k can be used as intermediate to get a shorter distance, let $P = u \rightsquigarrow k \rightsquigarrow v$ be a shortest path using intermediate vertices 1 to k .
 - ▶ Obviously, P cannot contain any cycle with positive weight.
 - ▶ Notice that P also cannot contain any negative cycle, as otherwise the shortest path $u \rightsquigarrow v$ would not be well-defined.

- Invariant: assuming that the shortest path $u \rightsquigarrow v$ is well-defined, $\text{dist}[u][v]$ corresponds to the minimum distance from u to v considering only intermediate vertices 1 to $k - 1$.
- Inductive step:
 - ▶ Let path Q be obtained by removing from P all cycles with 0 weight, so that Q has the same weight as P but no cycles.
 - ▶ Q must still contain vertex k exactly once, otherwise the previous value of $\text{dist}[u][v]$ would not be improved.
 - ▶ Let R be the part of Q that goes from u to k , and S be the remaining part of Q . Both R and S only contain intermediate nodes from 1 to $k - 1$.
 - ▶ By the optimality of subpaths in shortest paths, R and S must have distances $\text{dist}[u][k]$ and $\text{dist}[k][v]$, respectively.
 - ▶ Thus P and Q have weight equal to $\text{dist}[u][k] + \text{dist}[k][v]$. And this is exactly the value that $\text{dist}[u][v]$ would get updated to.

Negative cycles



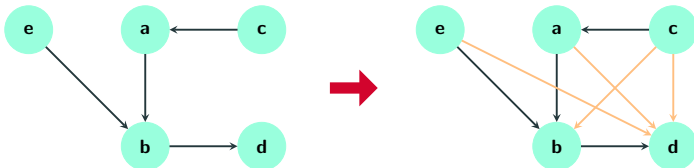
dist	1	2	3	4
1	0	-5	-3	-5
2	5	-1	1	-1
3	3	-3	-1	-3
4	4	-2	0	-2

- If there is a negative cycle, there will be a vertex v such that $dist[v][v]$ is negative.
- Look at the diagonal of the final matrix and return error if a negative value is found.
- **How could you modify the algorithm to return the paths?**

Transitive closure

Transitive closure of a graph

- Given a graph $G = (V, E)$, its transitive closure is another graph $G' = (V, E')$ that contains the same vertices V but contains an edge $u \rightarrow v$ if there is a path $u \rightsquigarrow v$ in G .



Warshall's transitive closure algorithm

Warshall's transitive closure algorithm

```
1: function TRANSITIVE_CLOSURE( $G = (V, E)$ )
2:    $connected[1..|V|][1..|V|] = \mathbf{false}$ 
3:    $connected[v][v] = \mathbf{true}$  for all vertices  $v$ 
4:    $connected[u][v] = \mathbf{true}$  for all edges  $e = (u, v)$  in  $E$ 
5:   for each vertex  $k = 1$  to  $|V|$  do
6:     for each vertex  $u = 1$  to  $|V|$  do
7:       for each vertex  $v = 1$  to  $|V|$  do
8:         // Invariant:  $connected[u][v]$  corresponds to the existence of path from  $u$  to  $v$  considering the intermediate vertices  $1$  to  $k - 1$ .
9:          $connected[u][v] = connected[u][v]$  or ( $connected[u][k]$  and  $connected[k][v]$ )
10:  return  $connected[1..|V|][1..|V|]$ 
```

- Time complexity: $\Theta(|V|^3)$
- Space complexity: $\Theta(|V|^2)$

- Course Notes: Chapter 8
- You can also check algorithms' textbooks for contents related to this lecture, e.g.:
 - ▶ CLRS: Sections 24.1 and 25.2
 - ▶ KT: Sections 6.8, 6.9 and 6.10
 - ▶ Rou: Chapter 18

Concluding remarks

- Take home message:
 - ▶ Dijkstra's algorithm works only for graphs with non-negative weights.
 - ▶ Bellman-Ford computes shortest paths in graphs with negative weights in $\Theta(|V| \cdot |E|)$ and can also detect the negative cycles that are reachable.
 - ▶ Floyd-Warshall algorithm computes all-pairs shortest paths and transitive closure in $\Theta(|V|^3)$.
- Things to do:
 - ▶ Go through recommended reading and make sure you understand why the algorithms are correct.
 - ▶ Implement Bellman-Ford and Floyd-Warshall algorithms.
- Coming up next: Network Flow